

PROGRAMAÇÃO MODULAR



XULAMBS PIZZA



ITERAÇÃO 4

PROF. JOÃO CARAM

PUC MINAS

BACHARELADO EM ENGENHARIA DE SOFTWARE

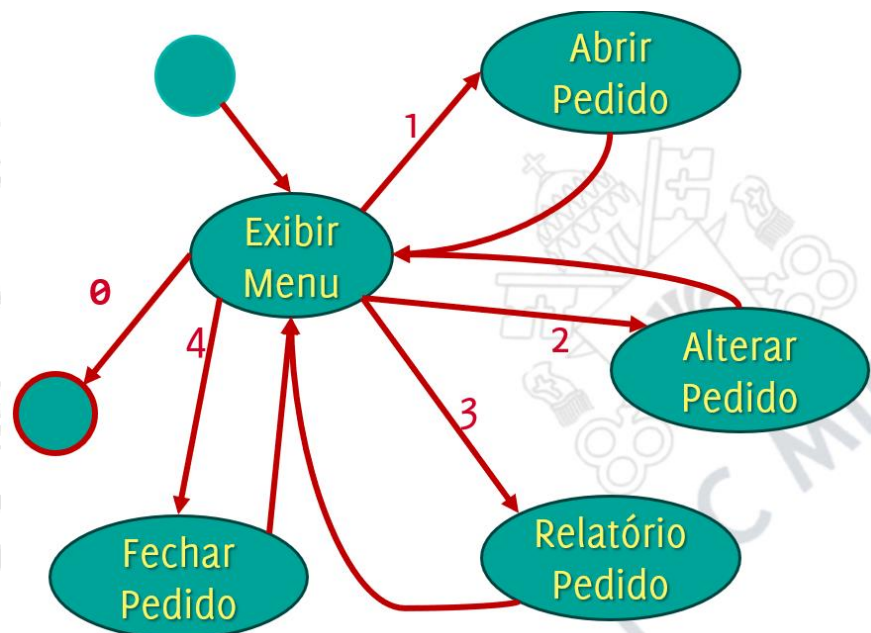
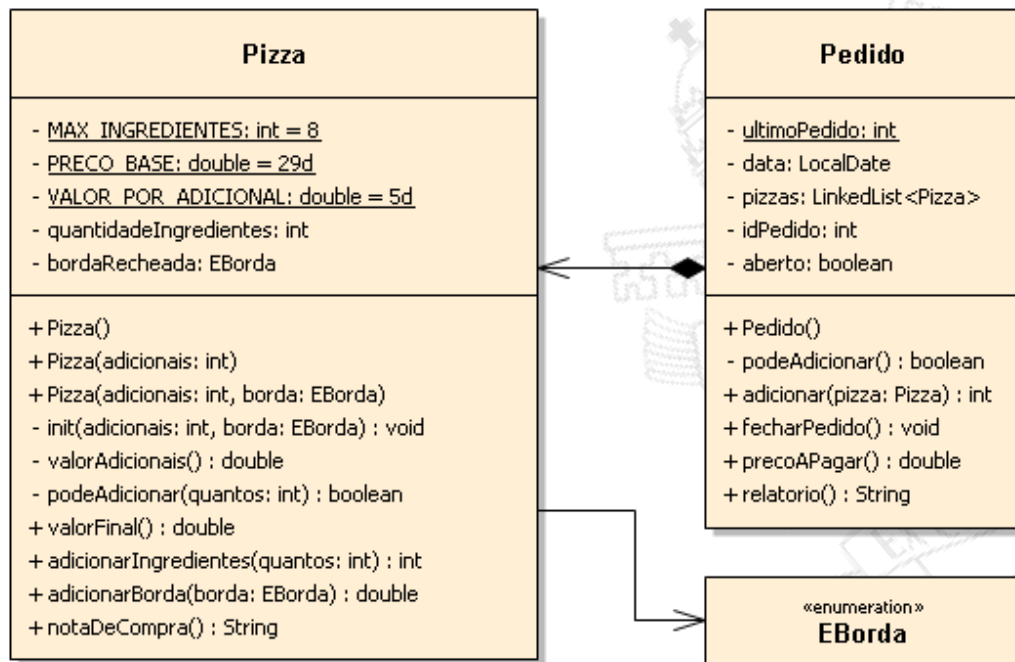
SISTEMA XULAMBS PIZZA

2

- O sistema da Xulambs Pizza está funcionando bem e assim estamos conquistando um cliente para nossa fábrica de software.

SISTEMA XULAMBS PIZZA

3



PIZZARIA E ENTREGAS

4

- A pizzaria quer ampliar seus negócios realizando pedidos para entrega de pizzas.



PIZZARIA E ENTREGAS

5

- Os pedidos para entrega terão limite de 8 pizzas. Eles incluem um valor de entrega no preço cobrado, que pode até ser R\$0, a depender da distância de entrega:
 - Até 4km: isento.
 - De 4,01 até 8km: R\$5
 - Acima de 8km: R\$8

A large, faint watermark of the PUC Minas logo is centered in the background. It features a shield with a star, a banner with the motto "EX CORDE ECCLESIAE", and the text "PUC Minas" below it.

{ Quero ver código!! }

QUEREMOS CÓDIGO, MAS ANTES...

7

- Análise!!!



QUEREMOS CÓDIGO, MAS ANTES...

8

■ Análise!!!

Pedido

- ultimoPedido: int
- data: LocalDate
- pizzas: LinkedList<Pizza>
- idPedido: int
- aberto: boolean

- + Pedido()
- podeAdicionar() : boolean
- + adicionar(pizza: Pizza) : int
- + fecharPedido() : void
- + precoAPagar() : double
- + relatorio() : String

QUEREMOS CÓDIGO, MAS ANTES...

9

■ Análise!!!

- *Pedido para entrega ou local;*
- *Taxa de entrega pela distância;*
- *Limite de 8 pizzas na entrega;*

Pedido

- ultimoPedido: int
- data: LocalDate
- pizzas: LinkedList<Pizza>
- idPedido: int
- aberto: boolean

- + Pedido()
- podeAdicionar() : boolean
- + adicionar(pizza: Pizza) : int
- + fecharPedido() : void
- + precoAPagar() : double
- + relatorio() : String

UM NOVO PEDIDO

10

■ Análise!!!

- *Pedido para entrega ou local;*
- *Taxa de entrega pela distância;*
- *Limite de 8 pizzas na entrega;*

Pedido

- ultimoPedido: int
- MAX_PIZZAS: int = 8
- TAXAS: double[] = [0,5,8]
- DISTANCIAS: double[] = [4,8,MAX]
- idPedido: int
- data: LocalDate
- distanciaEntrega: double
- paraEntrega: boolean
- pizzas: LinkedList<Pizza>
- aberto: boolean

- + Pedido()
- + Pedido(distancia: double)
- podeAdicionar() : boolean
- valorTaxa() : void
- + adicionar(pizza: Pizza) : int
- + fecharPedido() : void
- + precoAPagar() : double
- + relatorio() : String

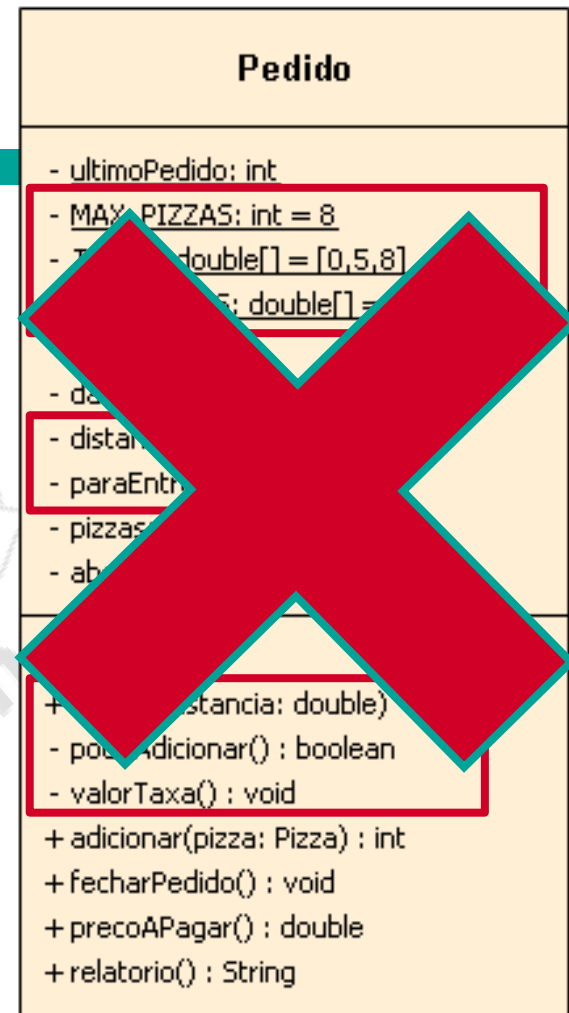
UM NOVO PEDIDO

11

■ Análise!!!

- *Pedido para entrega ou local;*
- *Taxa de entrega pela distância;*
- *Limite de 8 pizzas na entrega;*

Infelizmente, péssima ideia.



ANÁLISE (RUIM) ➡ CÓDIGO (RUIM)

```
private double valorTaxa() {  
    double taxa = 0;  
    if (paraEntrega) {  
        int pos = 0  
        while(distanciaEntrega > DISTANCIAS[pos])  
            pos++;  
        taxa = TAXAS[pos];  
    }  
    return taxa;  
}
```

ANÁLISE (RUIM) ➡ CÓDIGO (RUIM)

```
private double valorTaxa() {  
    double taxa = 0;  
    if (paraEntrega) {  
        int pos = 0  
        while(distanciaEntrega > DISTANCIAS[pos])  
            pos++;  
        taxa = TAXAS[pos];  
    }  
    return taxa;  
}
```

E a responsabilidade única?

E a coesão?

E se depois mudar a regra?

E se surgirem outros tipos de Pedido?

ANÁLISE (RUIM) ➡ CÓDIGO (RUIM)

```
private bool podeAdicionar() {  
    bool resposta = aberto;  
    if (paraEntrega)  
        resposta = aberto && (pizzas.size() < MAX_PIZZAS);  
    return resposta;  
}
```

ANÁLISE (RUIM) ➡ CÓDIGO (RUIM)

```
public String relatorio() {  
    StringBuilder relat ...;  
    if(paraEntrega)  
        relat.append(" - PARA ENTREGA\n");  
    else  
        relat.append(" - LOCAL\n");  
    ...  
    if(paraEntrega)  
        relat.append($" \nTAXA ENTREGA: .....");  
}
```

ANÁLISE (RUIM) ➡ CÓDIGO (RUIM)

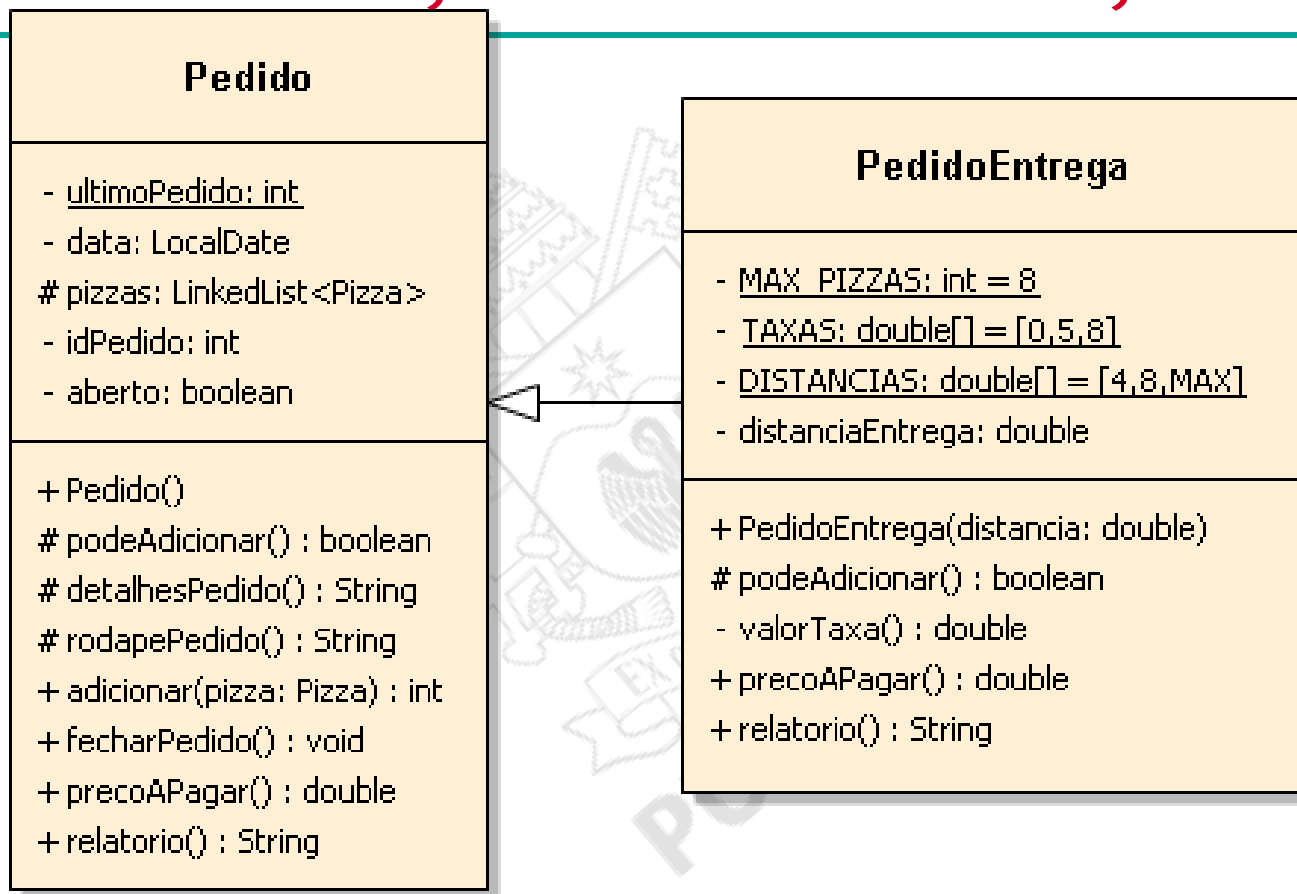
```
public String relatorio() {  
    StringBuilder relat ...;  
    if(paraEntrega)  
        relat.append(" - PARA ENTREGA\n");  
    else  
        relat.append(" - LOCAL\n");  
    ...  
    if(paraEntrega)  
        relat.append($" \nTAXA ENTREGA: .....");  
}
```

ESPECIALIZAÇÃO!

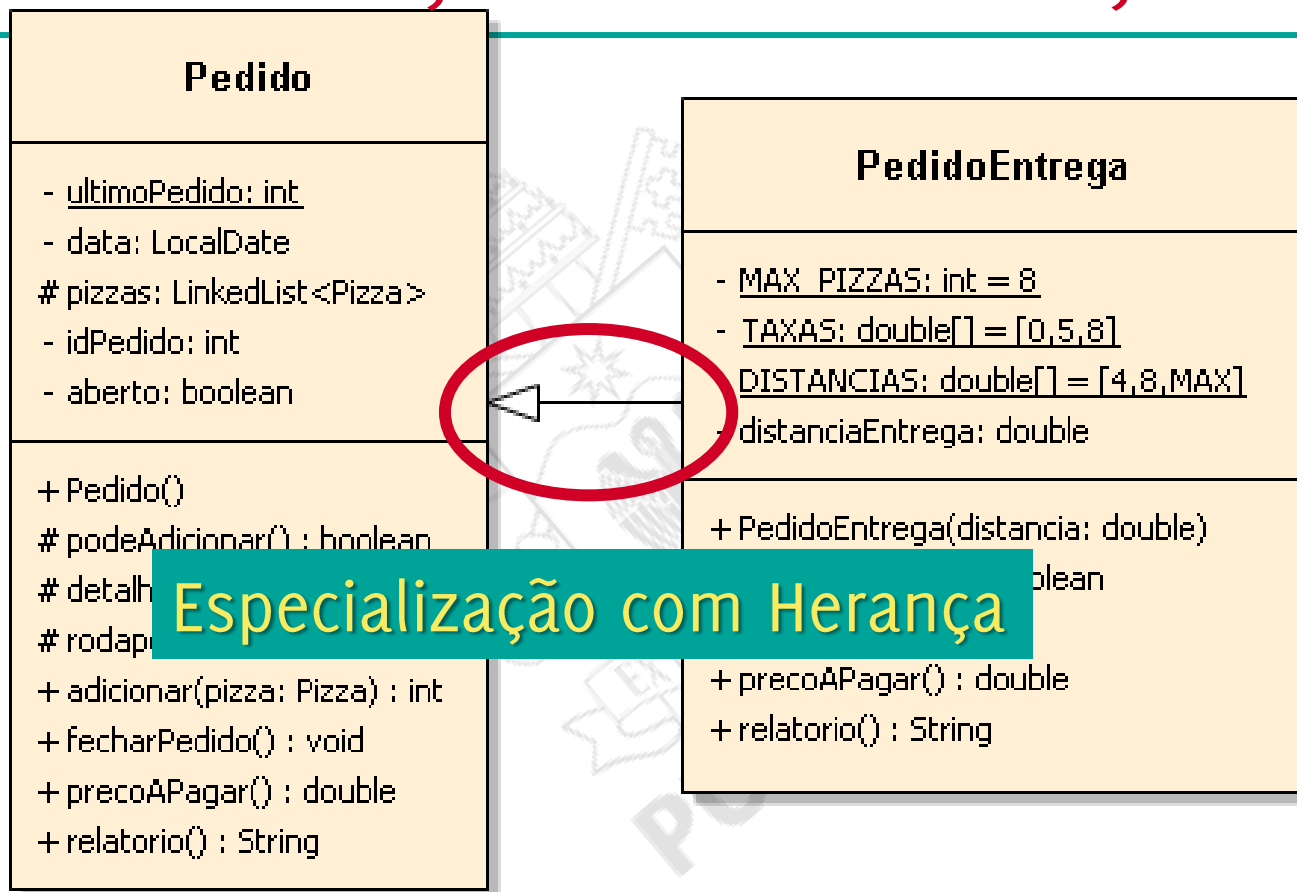
17

- Dois tipos de pedido: duas classes de negócio;
- Classe mãe: contém o que é comum aos dois tipos;
 - Não pode ser instanciada (abstrata)
- Classes filhas com suas regras particulares;
 - Dependência do construtor da classe mãe

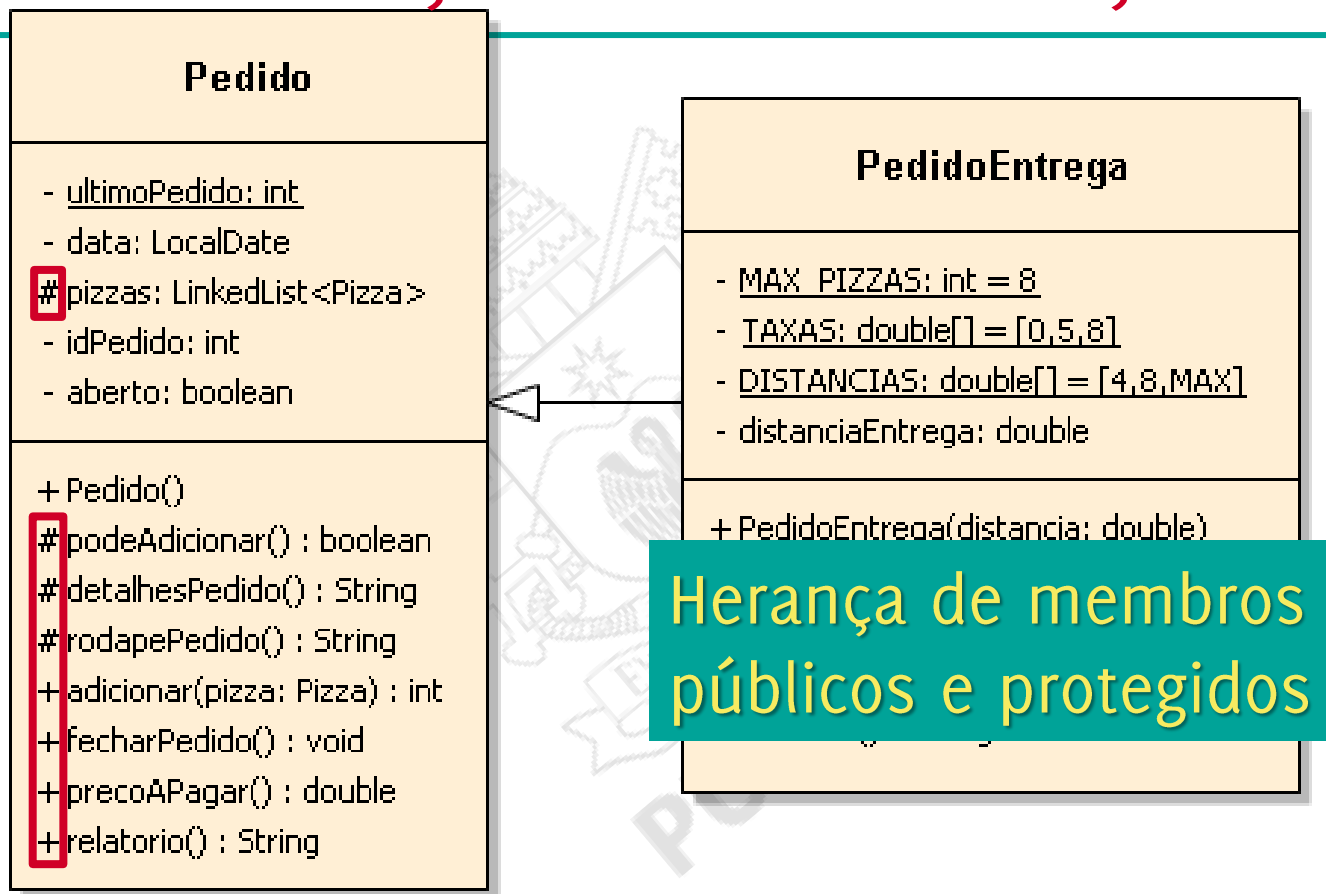
ESPECIALIZAÇÃO COM HERANÇA



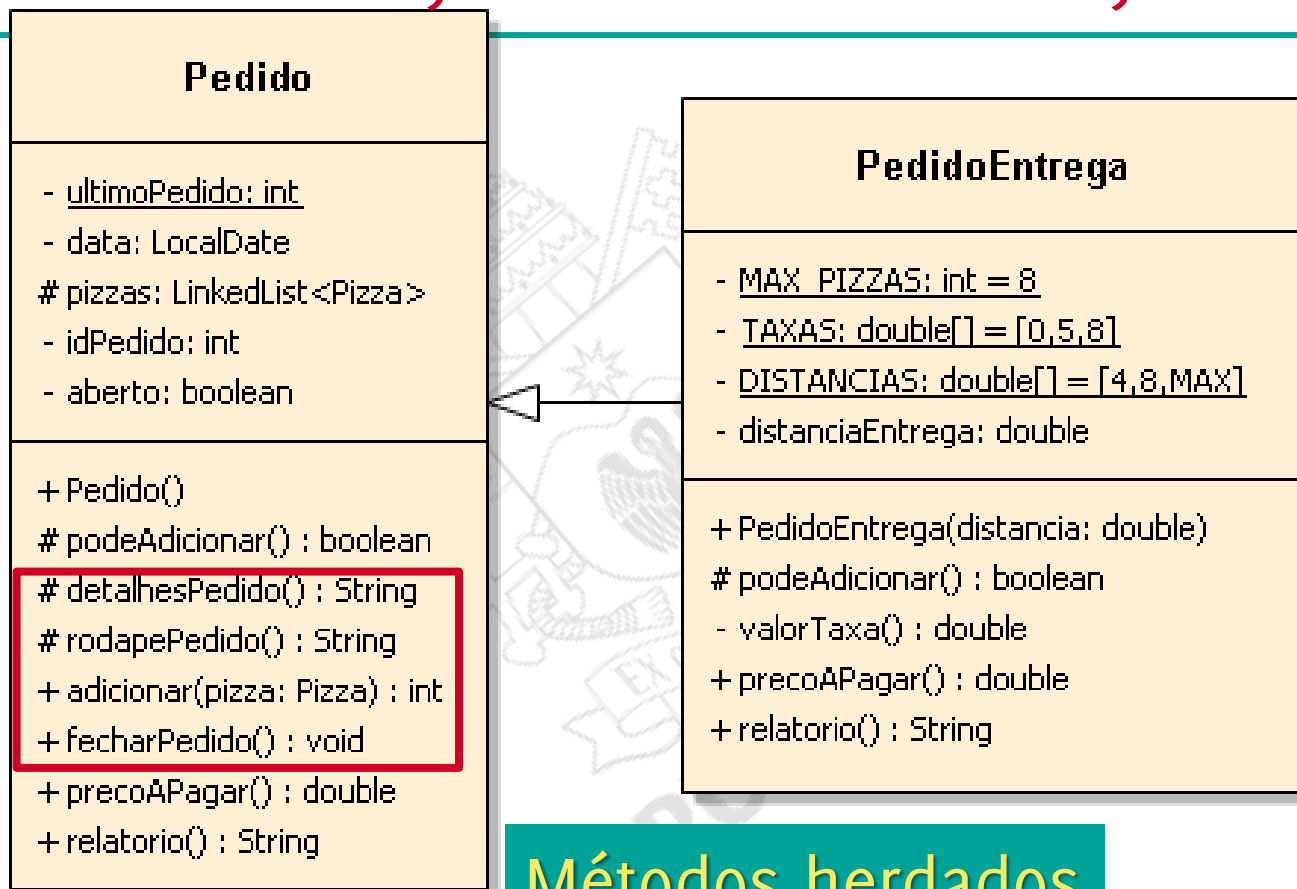
ESPECIALIZAÇÃO COM HERANÇA



ESPECIALIZAÇÃO COM HERANÇA

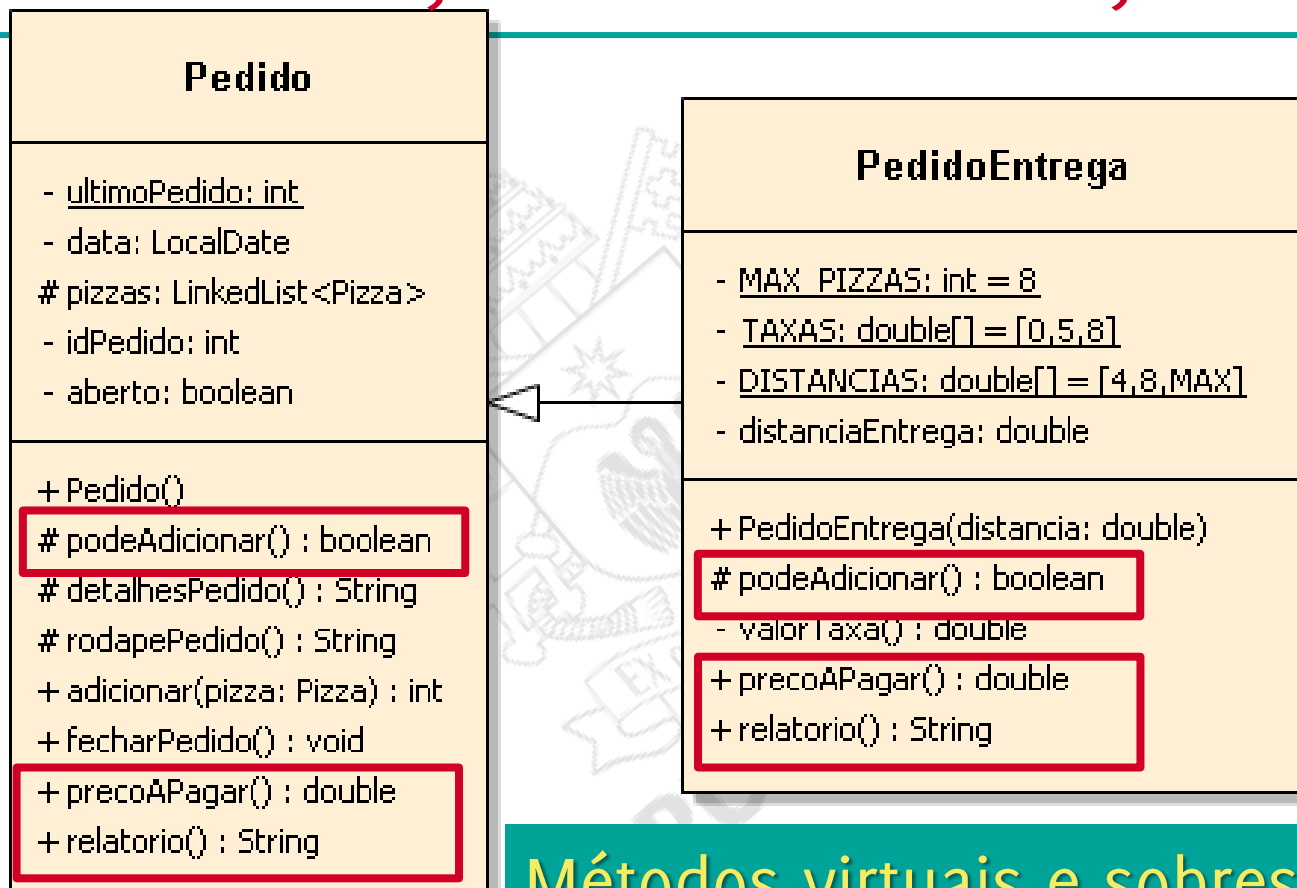


ESPECIALIZAÇÃO COM HERANÇA



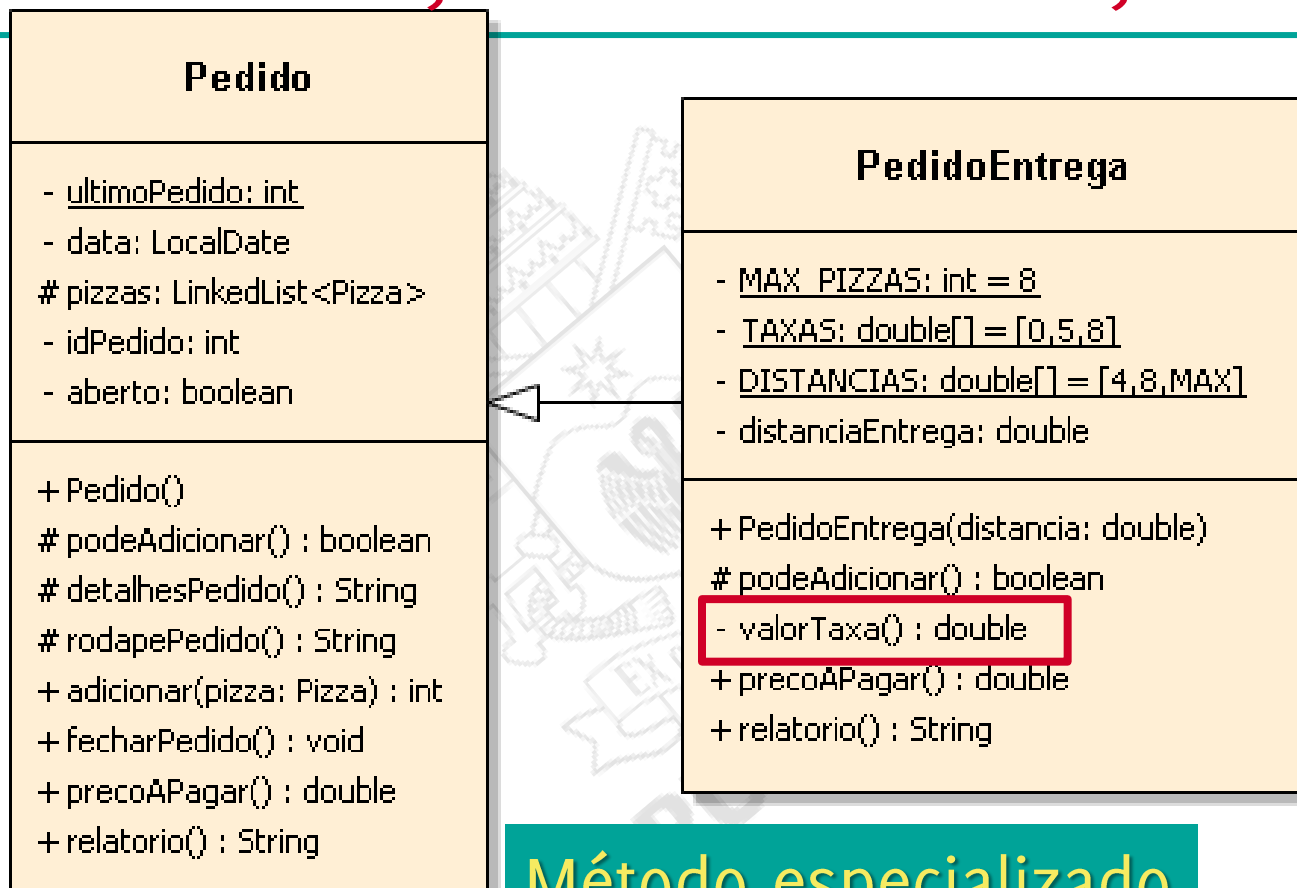
Métodos herdados

ESPECIALIZAÇÃO COM HERANÇA



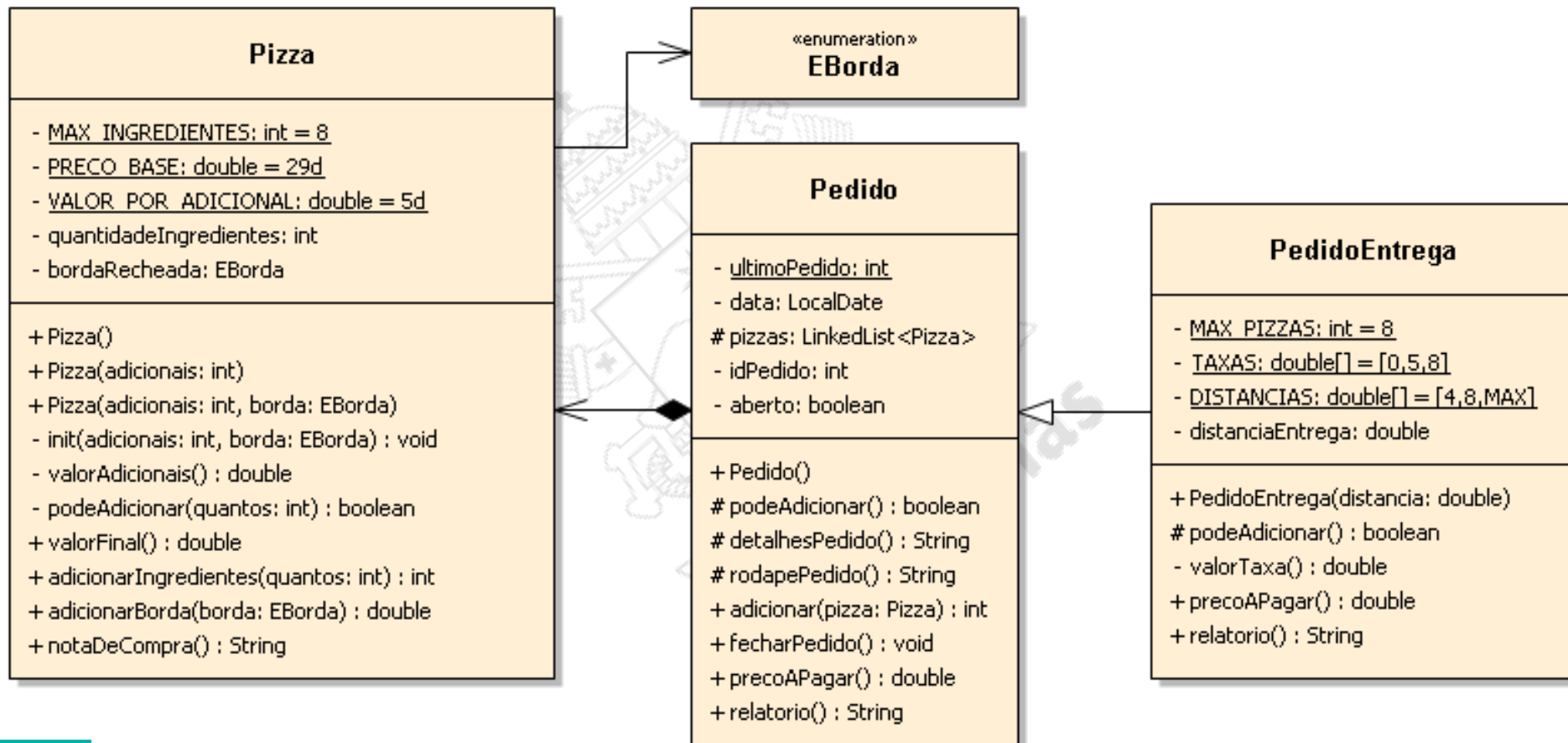
Métodos virtuais e sobrescritos

ESPECIALIZAÇÃO COM HERANÇA



Método especializado

ESPECIALIZAÇÃO COM HERANÇA

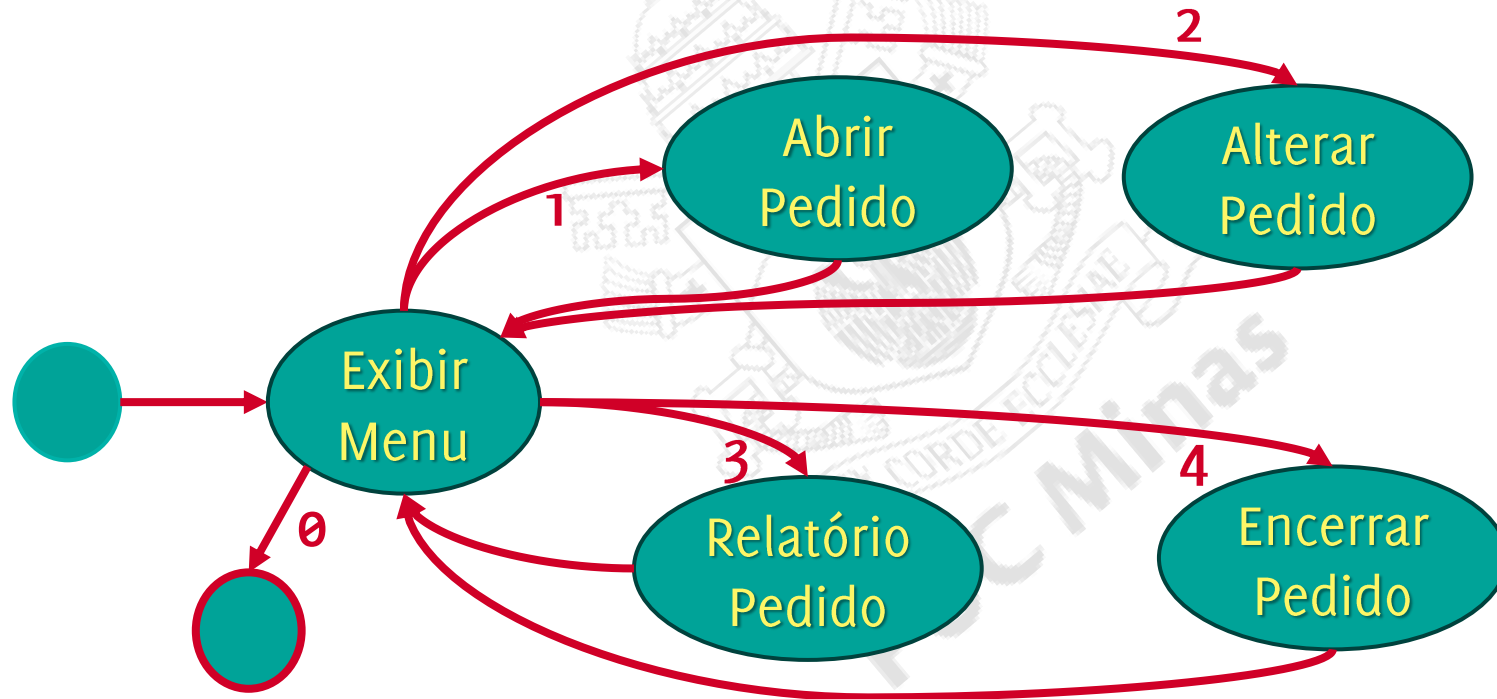


A large, faint watermark of the PUC Minas logo is centered in the background. It features a shield with a star, a banner with the motto "EX CORDE ECCLESIAE", and the text "PUC Minas" below it.

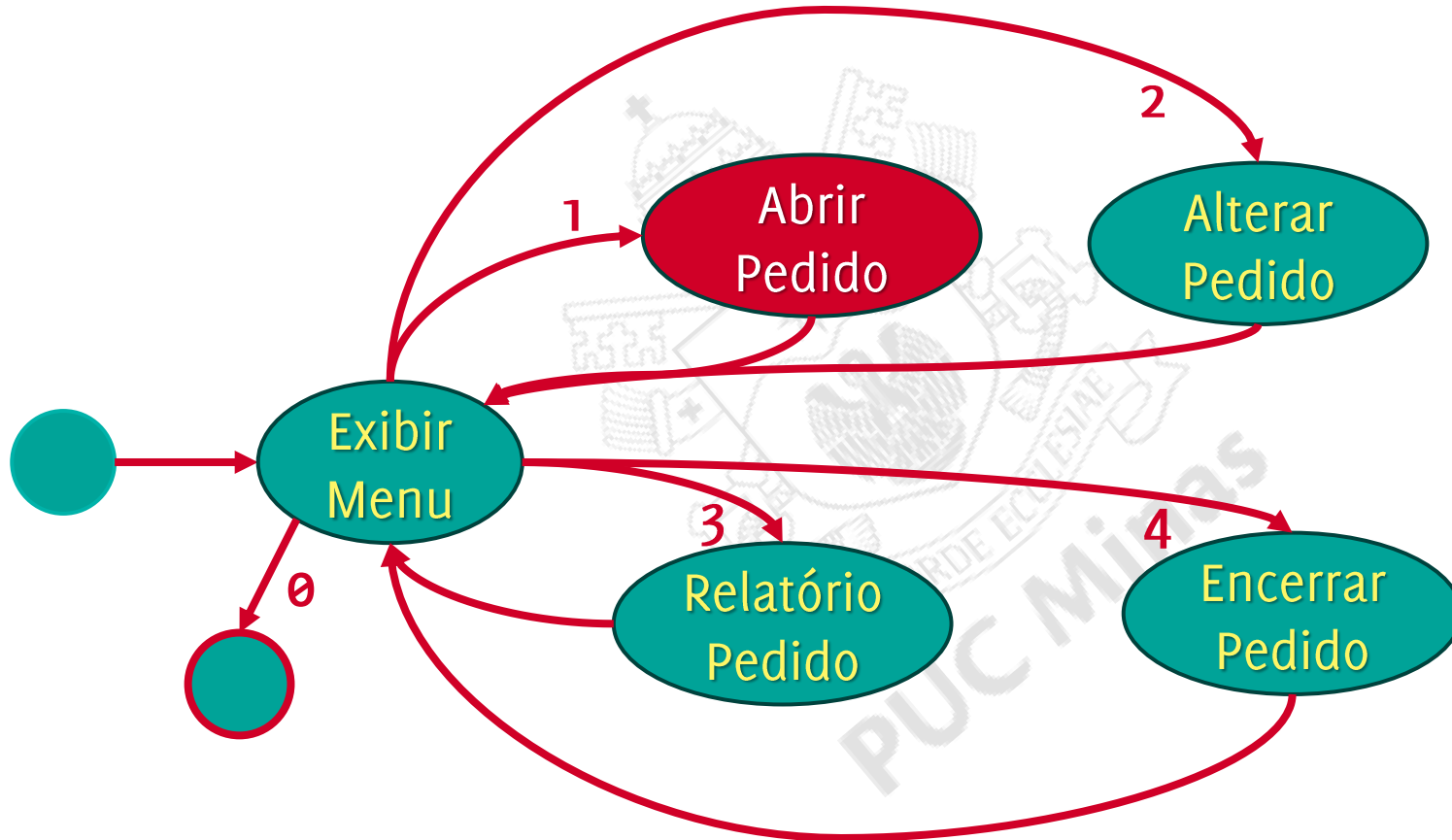
{ Quero ver código!! }

E O SISTEMA?

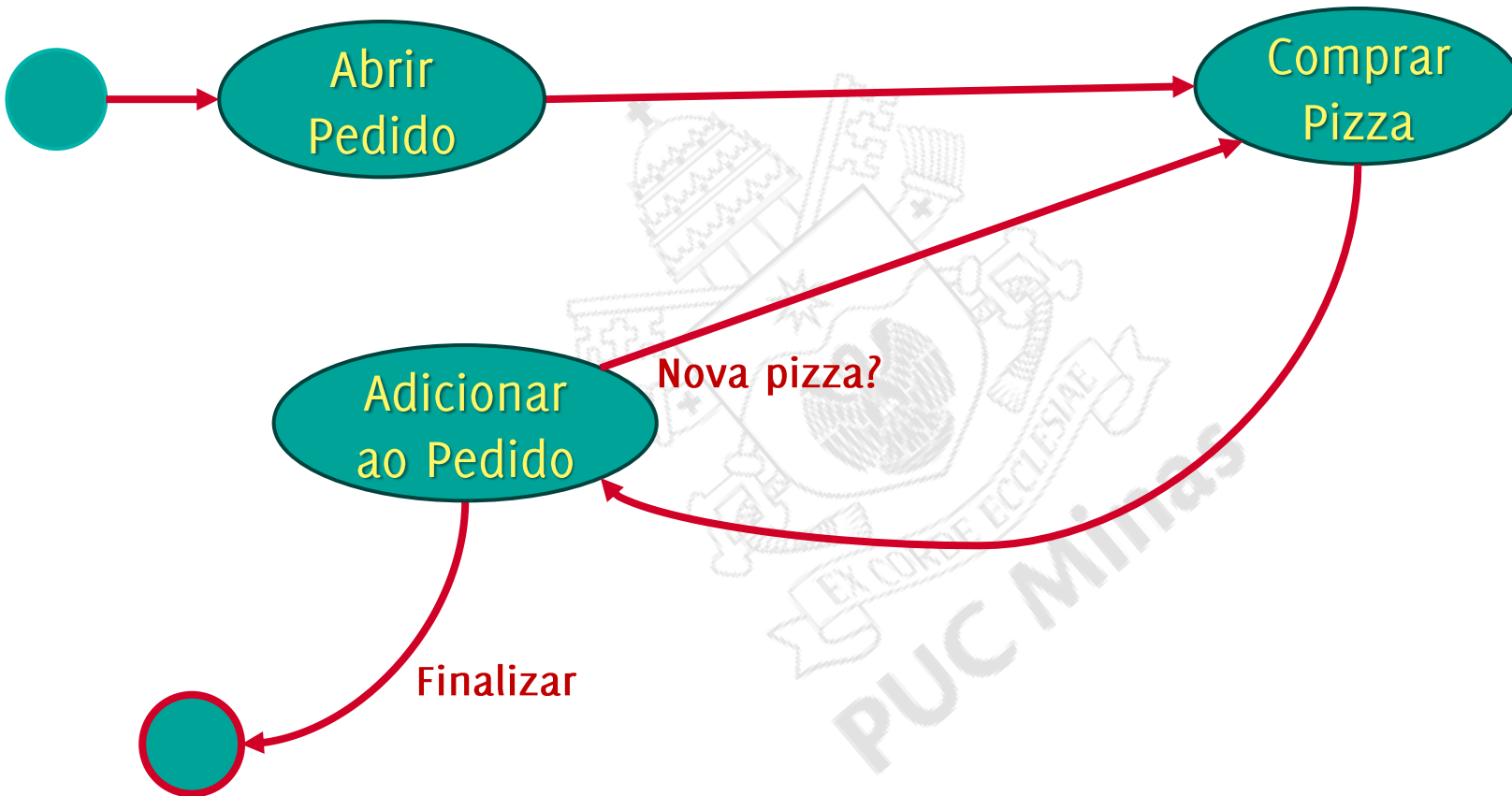
26



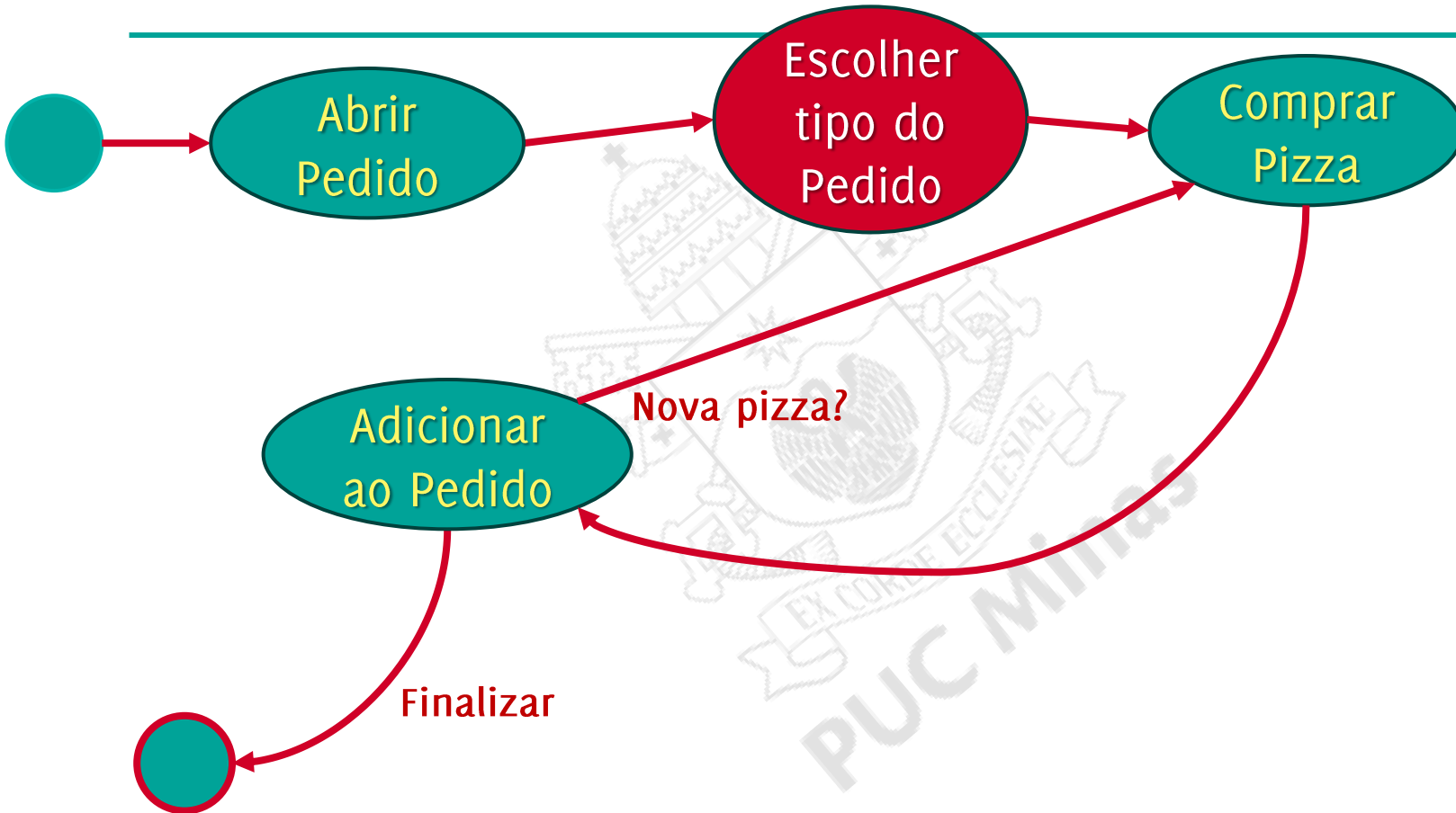
PROCESSO → SISTEMA



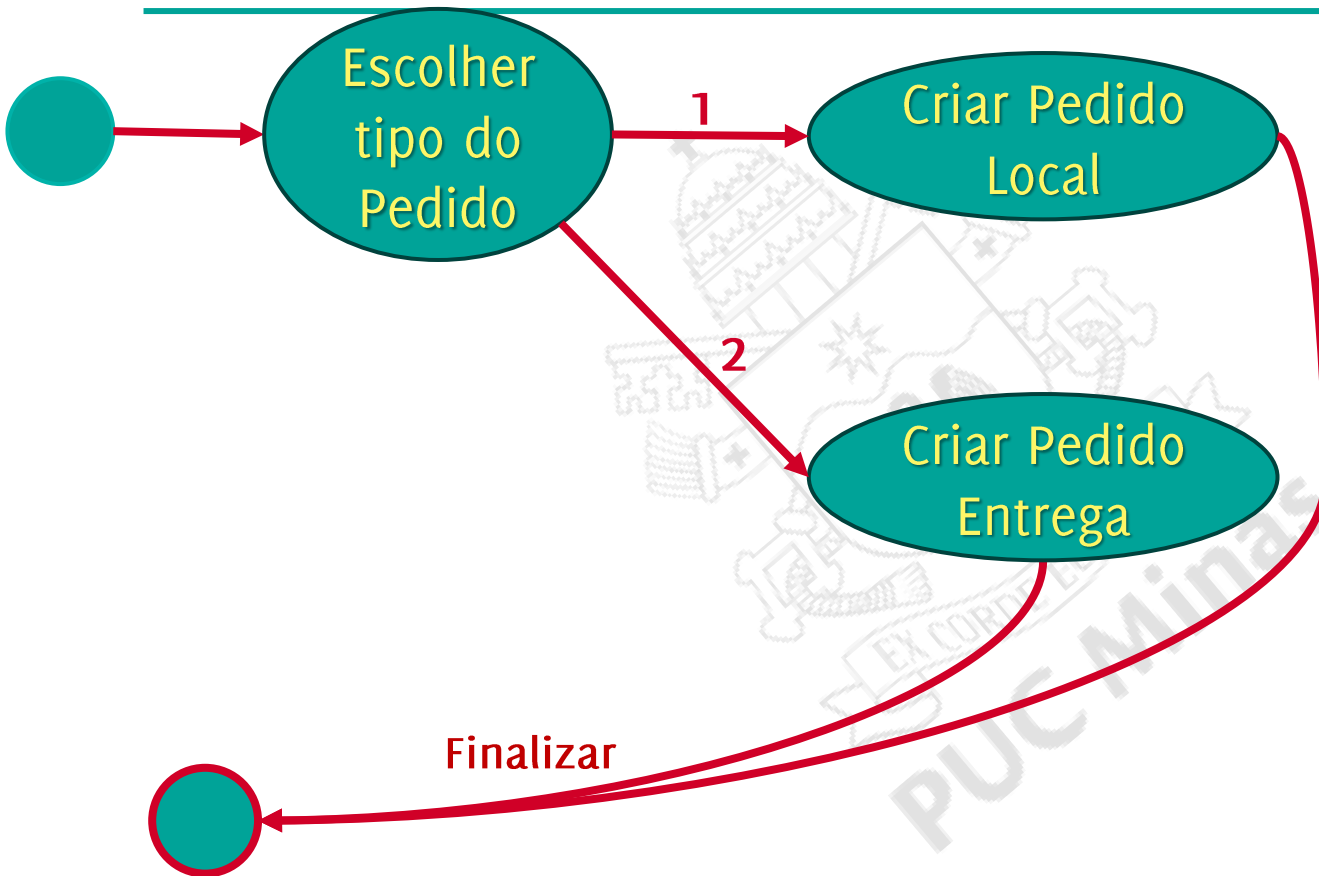
PROCESSO ➡ SISTEMA



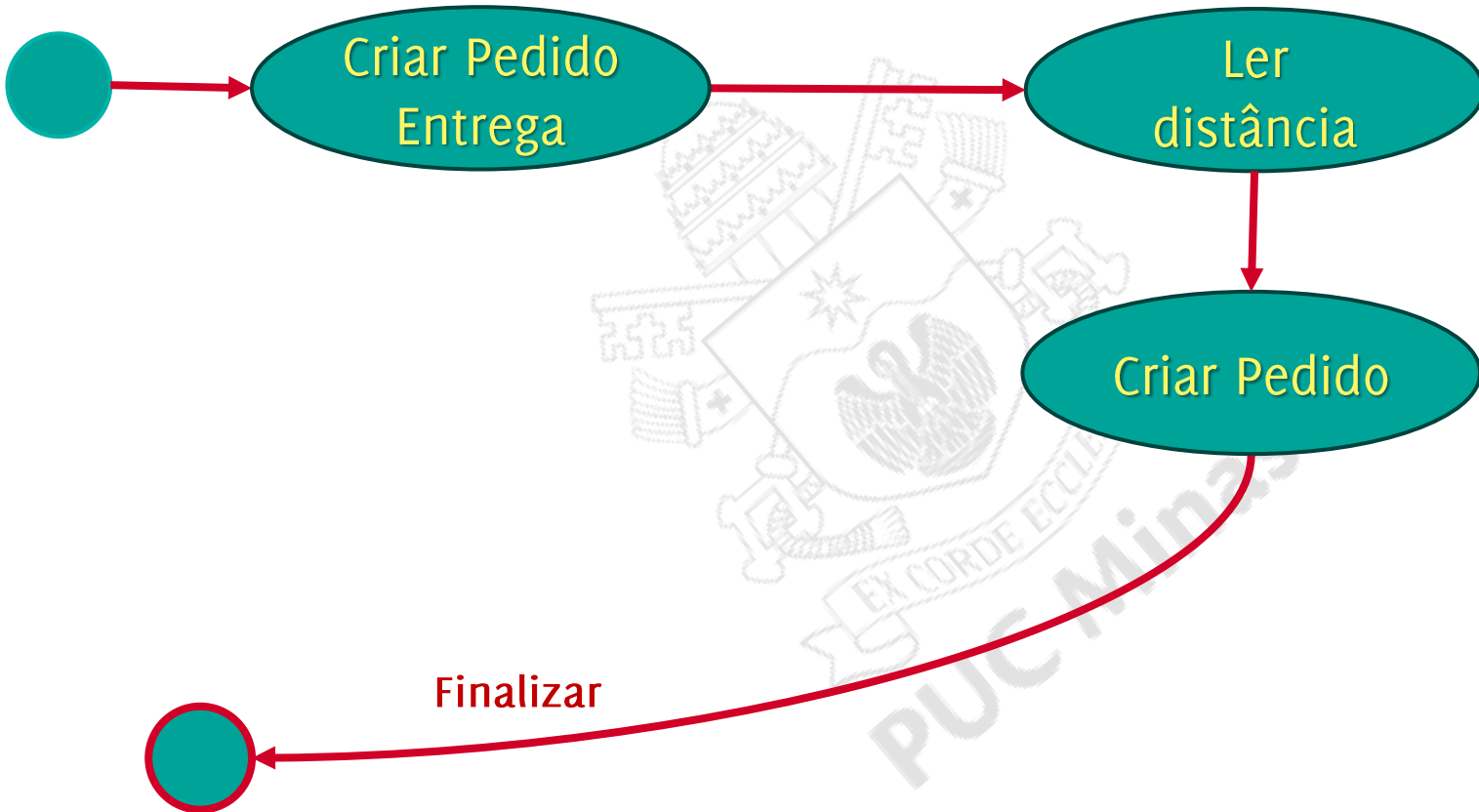
PROCESSO ➡ SISTEMA



PROCESSO → SISTEMA



PROCESSO → SISTEMA



{ Quero ver código!! }



OBRIGADO.

DÚVIDAS?